Seri Port (UART) Üzerinden Boot (BeagleBone Black)

Bu dokümanda BeagleBone Black'i **SD kart ve eMMC kullanmadan**, bütün boot dosyalarını bilgisayardan **seri port (UART0)** üzerinden göndererek açmayı inceleyeceğiz.

> **Not**
>
> SD karttan boot için: booting.md · U-Boot'a seri porttan dosya gönderme (`loady`) için: uboot_uEnv_ve_seri_yukleme.md

---

1. Dosya Transfer Protokolleri



Bilgisayardan karta dosya göndermenin iki yolu var:

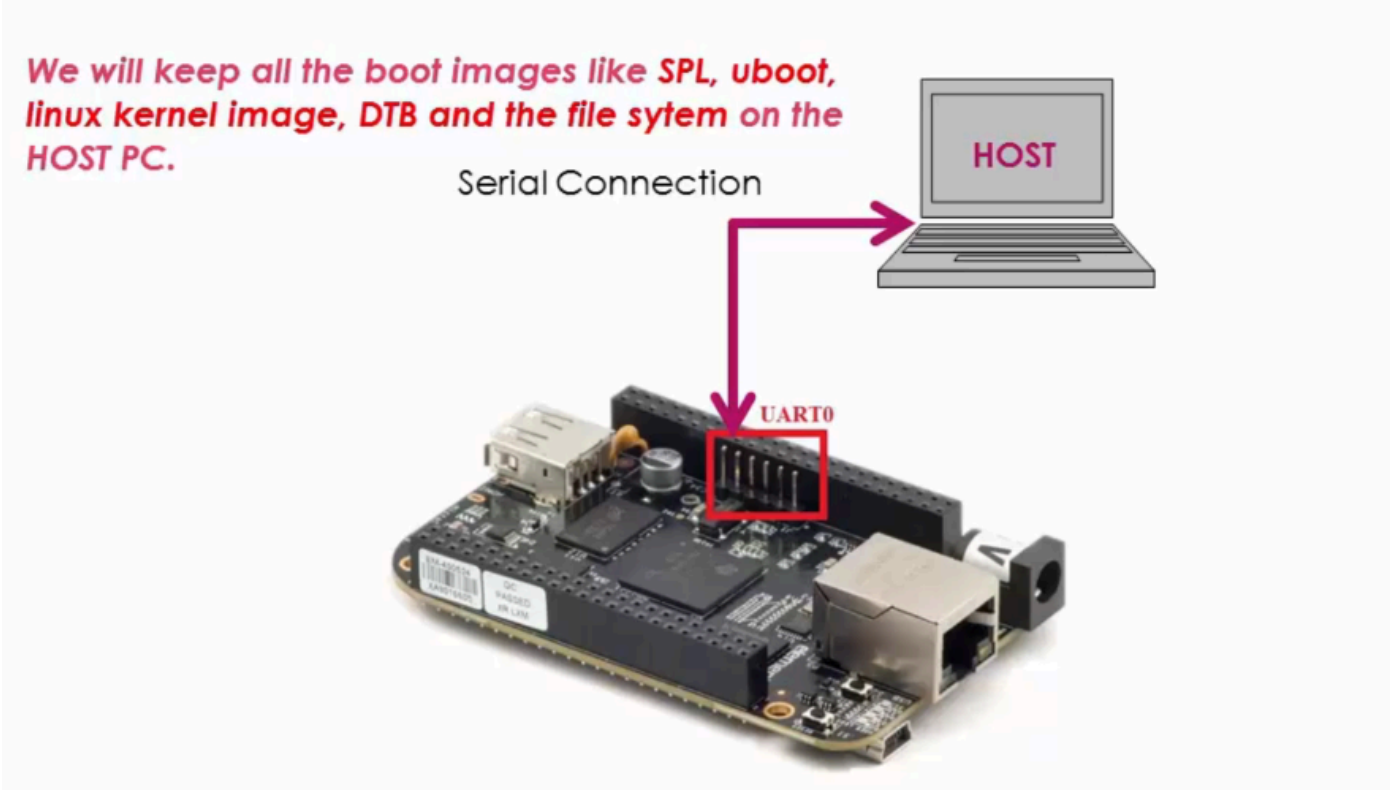
Bağlantı	Protokoller	Ne zaman kullanılır?
IP ağı (Ethernet)	TFTP, HTTP, FTP, SMTP...	Kartta çalışan bir ağ yığını (U-Boot veya Linux) varsa. Hızlıdır.
UART (seri port)	Xmodem, Ymodem, Zmodem, Kermit...	Ağ yokken veya kartta henüz hiçbir yazılım yokken. Yavaştır ama çok basittir.

UART sadece ham baytları taşır; "dosya nerede başlıyor, nerede bitiyor, veri bozuldu mu?" gibi bilgileri taşımaz. **Xmodem / Ymodem** gibi protokoller veriyi paketlere bölüp her pakete kontrol bilgisi (checksum / CRC) ekler, bozuk paketi tekrar istetir.

Protokol	Özellik
Xmodem	128 bayt (Xmodem-1K'da 1024 bayt) paketler. Dosya adı ve boyutu göndermez. AM335x'in **ROM kodu** bunu kullanır.
Ymodem	Xmodem'in geliştirilmiş hali: dosya adını ve boyutunu da gönderir. U-Boot'ta `loady` komutu.

---

2. Seri Boot Fikri

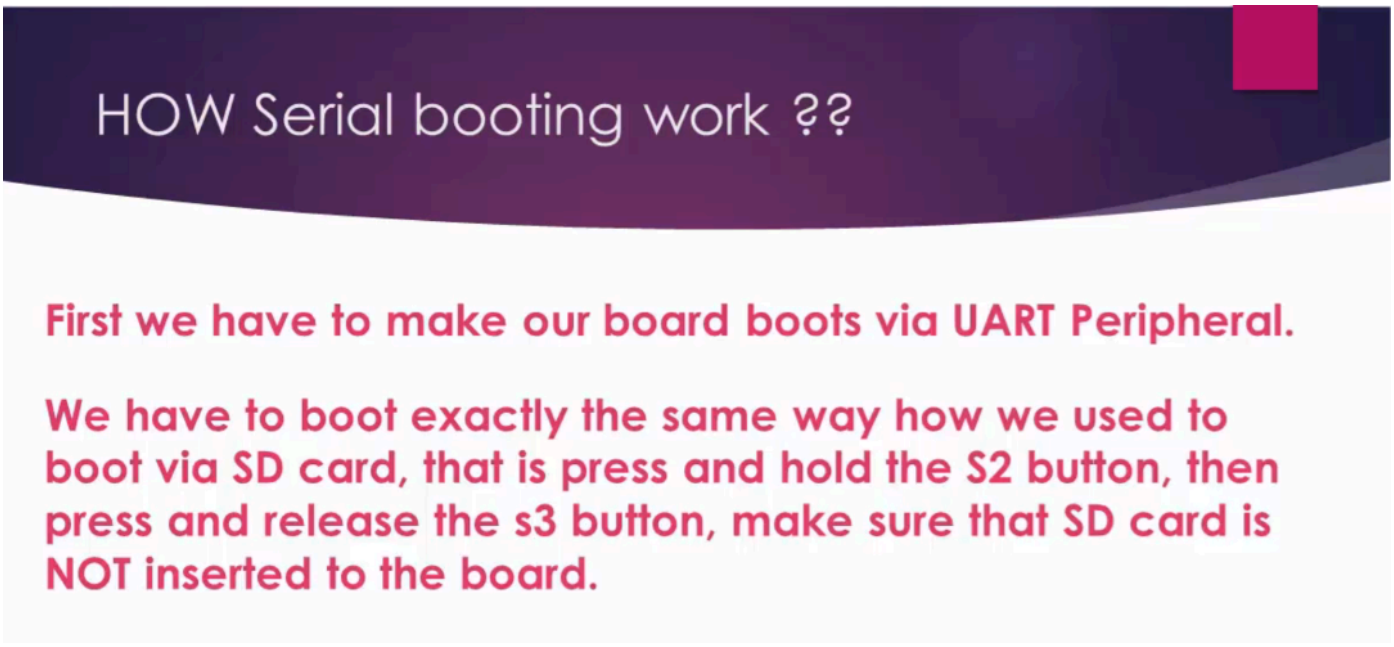


Bütün boot dosyaları — **SPL, U-Boot, Linux kernel (uImage), DTB ve dosya sistemi** — kartta değil, **bilgisayarda (HOST)** durur. Kart ile bilgisayar arasındaki tek bağlantı, J1 header'daki **UART0** pinlerine bağlı USB-TTL seri kablodur. Kart açılırken her dosyayı bu kablo üzerinden sırayla göndeririz.

Bu yöntem şu durumlarda işe yarar: - SD kart ve eMMC boşsa veya içindeki bootloader bozulduysa (kartı "kurtarmak" için). - Yeni derlenmiş bir SPL / U-Boot'u karta yazmadan denemek için.

---

3. Kartı UART Boot Moduna Sokmak

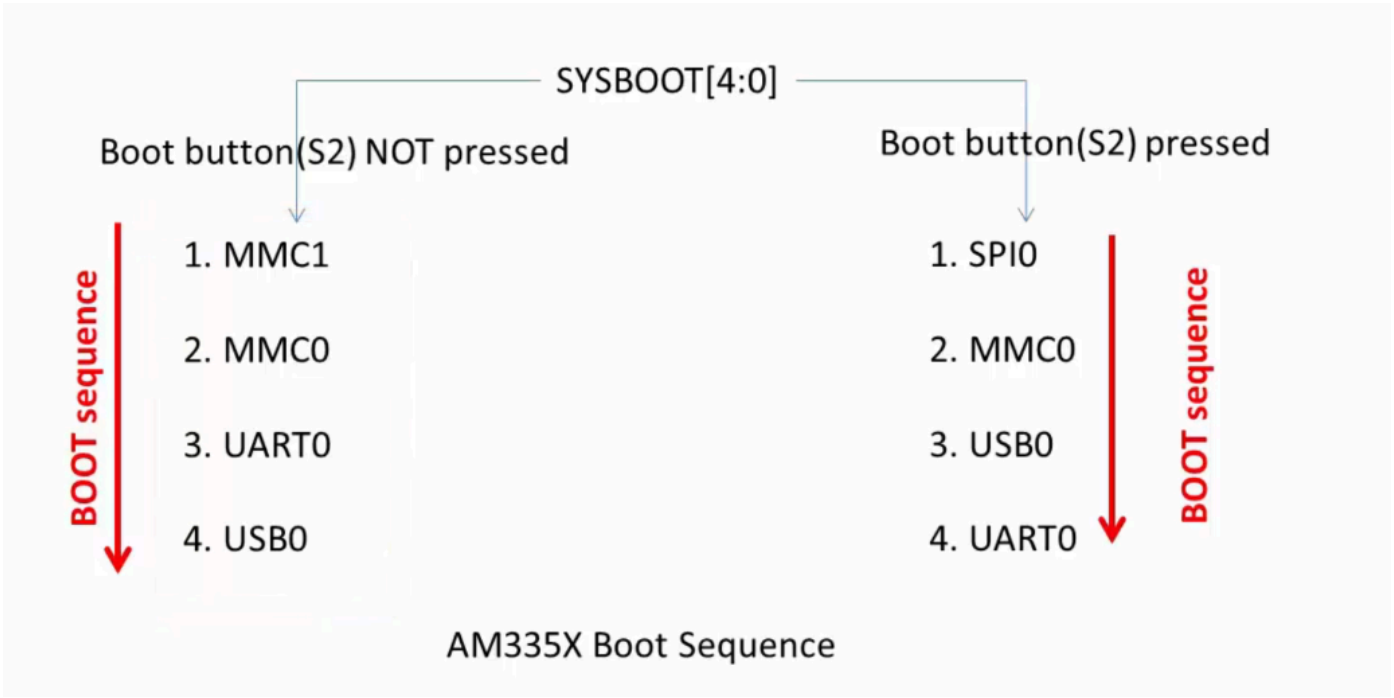


Kartı UART'tan boot ettirmek için SD karttan boot ederken yaptığımızın aynısını yaparız:

1. **S2 (BOOT)** butonuna basılı tut,
2. **S3 (POWER)** butonuna basıp bırak (veya beslemeyi tak),
3. Boot başladıktan sonra S2'yi bırak.

Tek fark: **SD kart takılı olmamalı.** Takılıysa ROM kodu boot dosyalarını SD karttan bulur ve UART'a hiç sıra gelmez.

AM335x boot sırası



AM335x'in içindeki ROM kodu, açılışta `SYSBOOT[4:0]` pinlerine bakarak hangi çevre birimlerini hangi sırayla deneyeceğine karar verir. BBB'de S2 butonu bu pinlerden birinin değerini değiştirir:

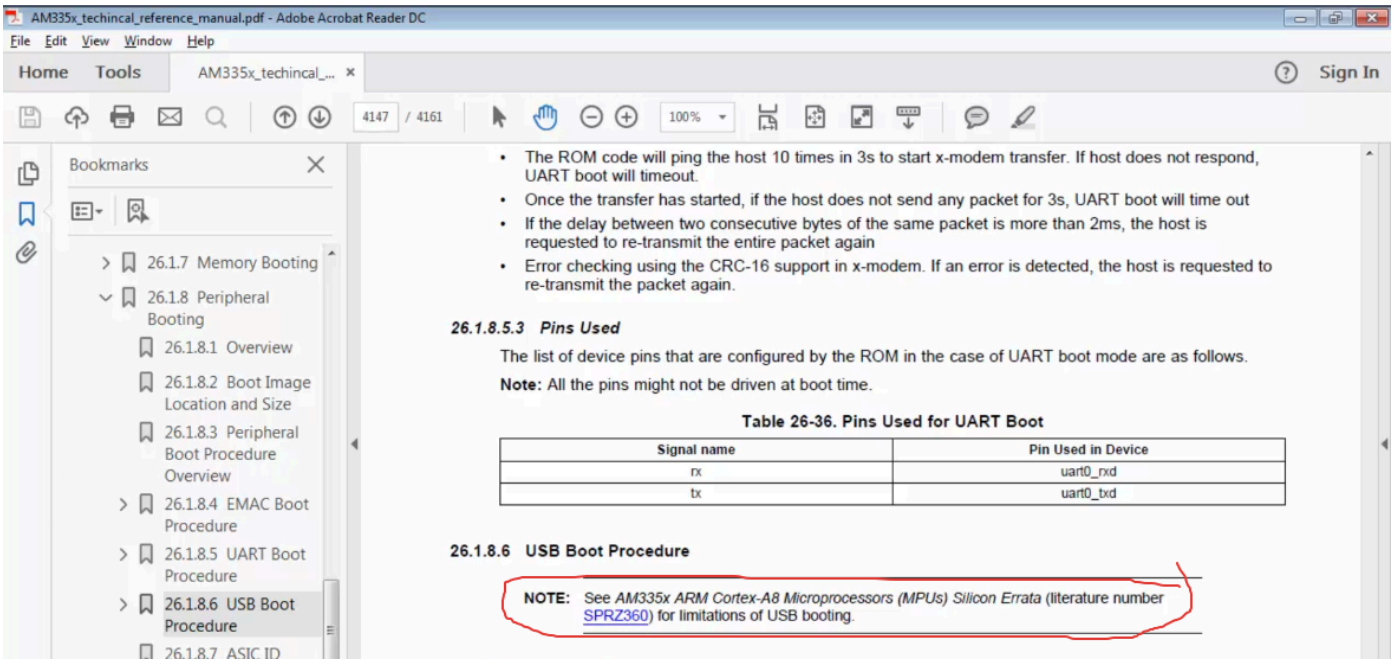
Sıra	S2 basılı **değil**	S2 basılı
1	MMC1 (eMMC)	SPI0
2	MMC0 (SD kart)	MMC0 (SD kart)
3	UART0	**USB0**
4	USB0	**UART0**

S2 basılı bir şekilde boot işlemi yaparsak ROM kodu sırayla her çevre biriminde geçerli bir boot image'ı olup olmadığını kontrol eder:

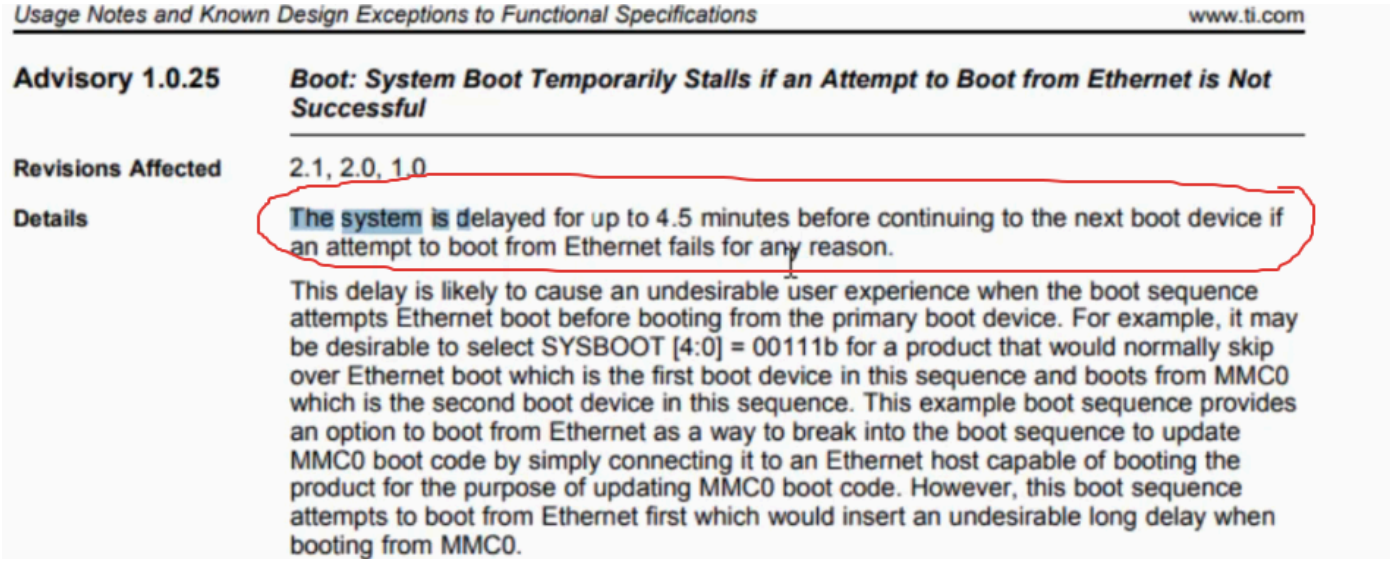
- **SPI0**: BBB'de SPI flash yok → hemen geçer.
- **MMC0**: SD kart takılı değil → hemen geçer.
- **USB0**: Burada farklı bir durum oluşuyor (aşağıda).
- **UART0**: Bizim istediğimiz yer.

USB0 sorunu: 4.5 dakikalık bekleme

Eğer USB kablosu bu sırada karta takılıysa ve image dosyası USB üzerinden gönderilmiyorsa, ROM kodu bir sonraki boot aygıtı olan UART0'a geçmeden önce **4.5 dakikaya kadar** bekliyor. Bu süre dolduktan sonra UART0'a geçiyor.



AM335x Technical Reference Manual'ın (TRM) **26.1.8.6 USB Boot Procedure** bölümünde, USB boot'un kısıtlamaları için **Silicon Errata (SPRZ360)** belgesine bakılması söyleniyor:



Usage Notes and Known Design Exceptions to Functional Specifications www.ti.com

Advisory 1.0.25 ***Boot: System Boot Temporarily Stalls if an Attempt to Boot from Ethernet is Not Successful***

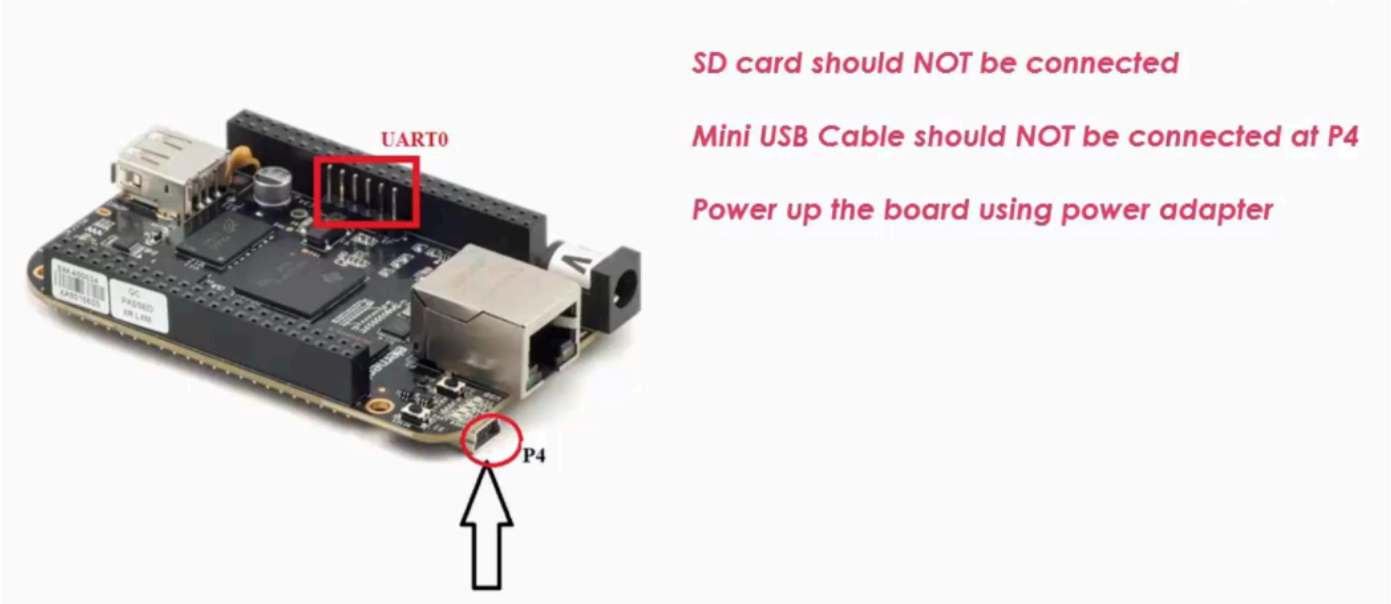
Revisions Affected 2.1, 2.0, 1.0

Details The system is delayed for up to 4.5 minutes before continuing to the next boot device if an attempt to boot from Ethernet fails for any reason.

This delay is likely to cause an undesirable user experience when the boot sequence attempts Ethernet boot before booting from the primary boot device. For example, it may be desirable to select SYSBOOT [4:0] = 00111b for a product that would normally skip over Ethernet boot which is the first boot device in this sequence and boots from MMC0 which is the second boot device in this sequence. This example boot sequence provides an option to boot from Ethernet as a way to break into the boot sequence to update MMC0 boot code by simply connecting it to an Ethernet host capable of booting the product for the purpose of updating MMC0 boot code. However, this boot sequence attempts to boot from Ethernet first which would insert an undesirable long delay when booting from MMC0.

Errata'daki **Advisory 1.0.25**'te Ethernet denilmiş, ancak bu durum USB için de geçerli. Çünkü AM335x'in ROM kodu USB boot'u, USB kablosu üzerinden **sanal bir Ethernet bağlantısı (RNDIS)** kurarak yapar. Yani USB boot aslında "USB üzerinden Ethernet boot"tur ve aynı 4.5 dakikalık beklemeye takılır.

Çözüm: Kartı harici adaptörle beslemek



Peki bu bekleme süresinden nasıl kurtulabiliriz? Beslemeyi **P4 mini USB portundan değil**, harici bir **5V - 2A adaptör** yardımıyla (DC jack) yapacağız. USB kablosu takılı olmayınca ROM kodu USB0'da hiçbir şey bulamaz, hiç beklemeden UART0 sekansına geçer ve image yüklemesini oradan yaparız.

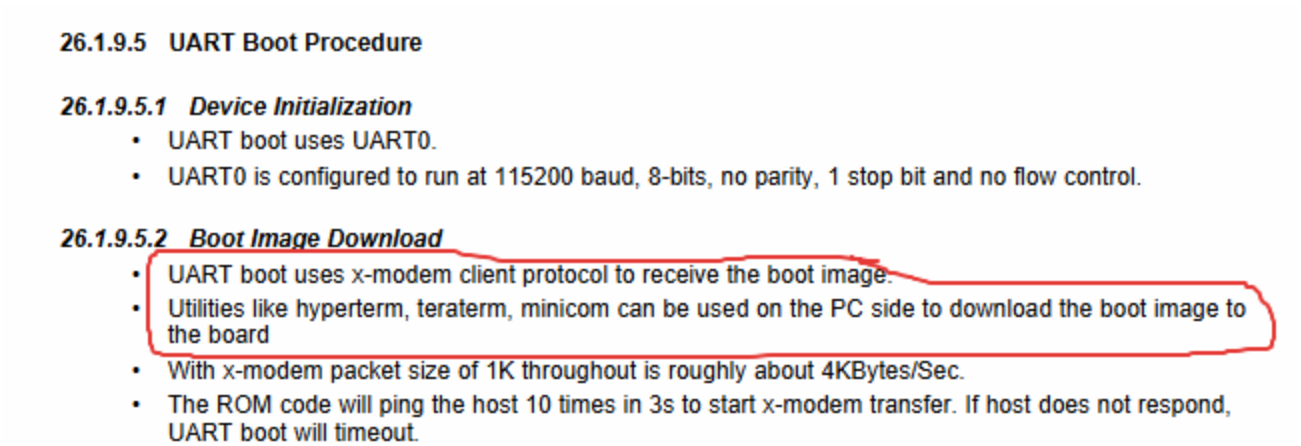
Koşul	Neden?
SD kart takılı olmamalı	ROM, UART0'dan önce MMC0'ı dener; SD kartta MLO varsa oradan açılır.
P4 (mini USB) kablosu takılı olmamalı	Takılıysa USB0 boot denenir ve 4.5 dakika beklenir.

Koşul	Neden?
Kart DC jack'ten 5V adaptörle beslenmeli	USB kablosu çıkınca kartın beslemesi başka yerden gelmeli.
UART0 seri kablosu bağlı olmalı	Bütün dosyalar buradan gidecek.

> **Uyarı**
>
> Seri kablonun (USB-TTL) **VCC / 5V** ucunu karta bağlama. J1 header'da sadece **GND, RX ve TX** bağlanır; kart zaten adaptörden besleniyor.

---

4. ROM Kodunun UART Boot Davranışı



26.1.9.5 UART Boot Procedure

26.1.9.5.1 Device Initialization

- UART boot uses UART0.
- UART0 is configured to run at 115200 baud, 8-bits, no parity, 1 stop bit and no flow control.

26.1.9.5.2 Boot Image Download

- UART boot uses x-modem client protocol to receive the boot image.
- Utilities like hyperterm, teraterm, minicom can be used on the PC side to download the boot image to the board
- With x-modem packet size of 1K throughout is roughly about 4KBytes/Sec.
- The ROM code will ping the host 10 times in 3s to start x-modem transfer. If host does not respond, UART boot will timeout.

TRM'nin **26.1.9.5 UART Boot Procedure** bölümüne göre:

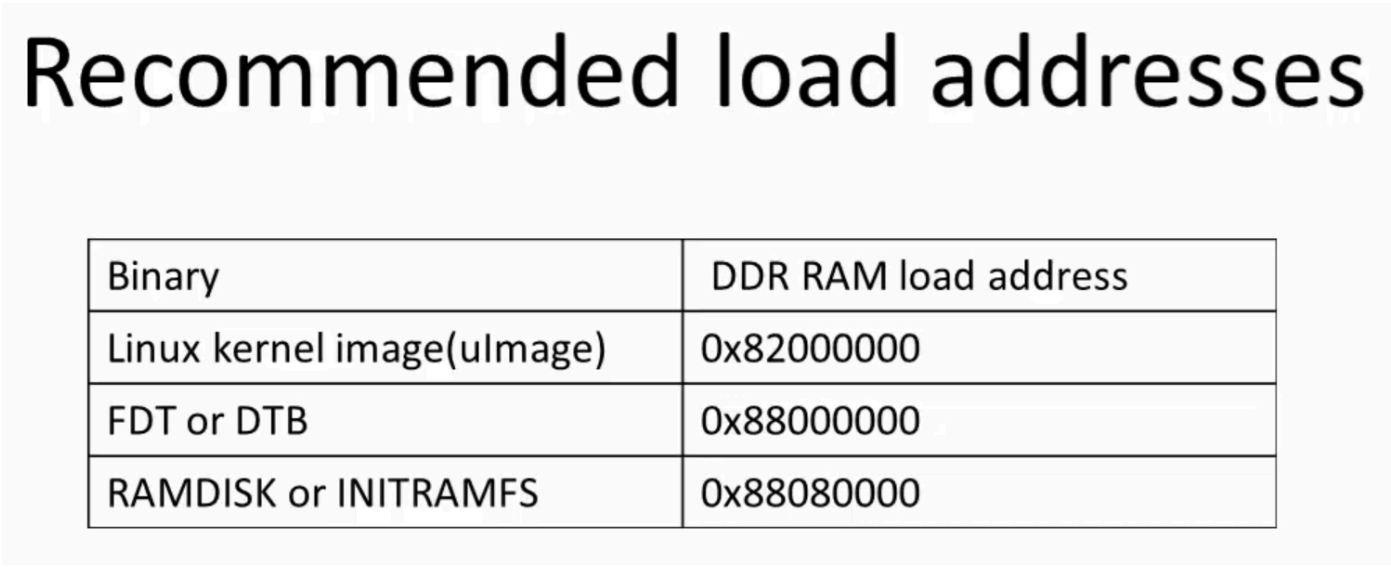
Konu	Değer
Kullanılan port	**UART0** (pinler: `uart0_rxd`, `uart0_txd`)
Ayarlar	**115200 baud, 8 bit, parity yok, 1 stop bit, flow control yok** (115200 8N1)
Protokol	**Xmodem** (ROM kodu Xmodem istemcisi olarak dosyayı alır)
Bilgisayar tarafı	HyperTerminal, Tera Term, **Minicom** gibi programlar kullanılabilir
Hız	1K paketlerle yaklaşık **4 KB/s**
Zaman aşımı	ROM, 3 saniye içinde 10 kez bilgisayarı "dürter" (ekrana `C` karakteri basar). Cevap gelmezse veya transfer başladıktan sonra 3 saniye paket gelmezse UART boot iptal olur.

Konu	Değer
Hata kontrolü	Xmodem'in CRC-16 desteği; bozuk paket tekrar istenir

> **İpucu**
>
> Minicom ekranında art arda `CCCC...` görmek, kartın UART boot moduna girip **dosya beklediği** anlamına gelir. 3 saniyelik süre kısa olduğu için Minicom'da gönderme menüsünü (`Ctrl-A` → `S` → **xmodem**) önceden hazırla.

---

5. Önerilen RAM Yükleme Adresleri



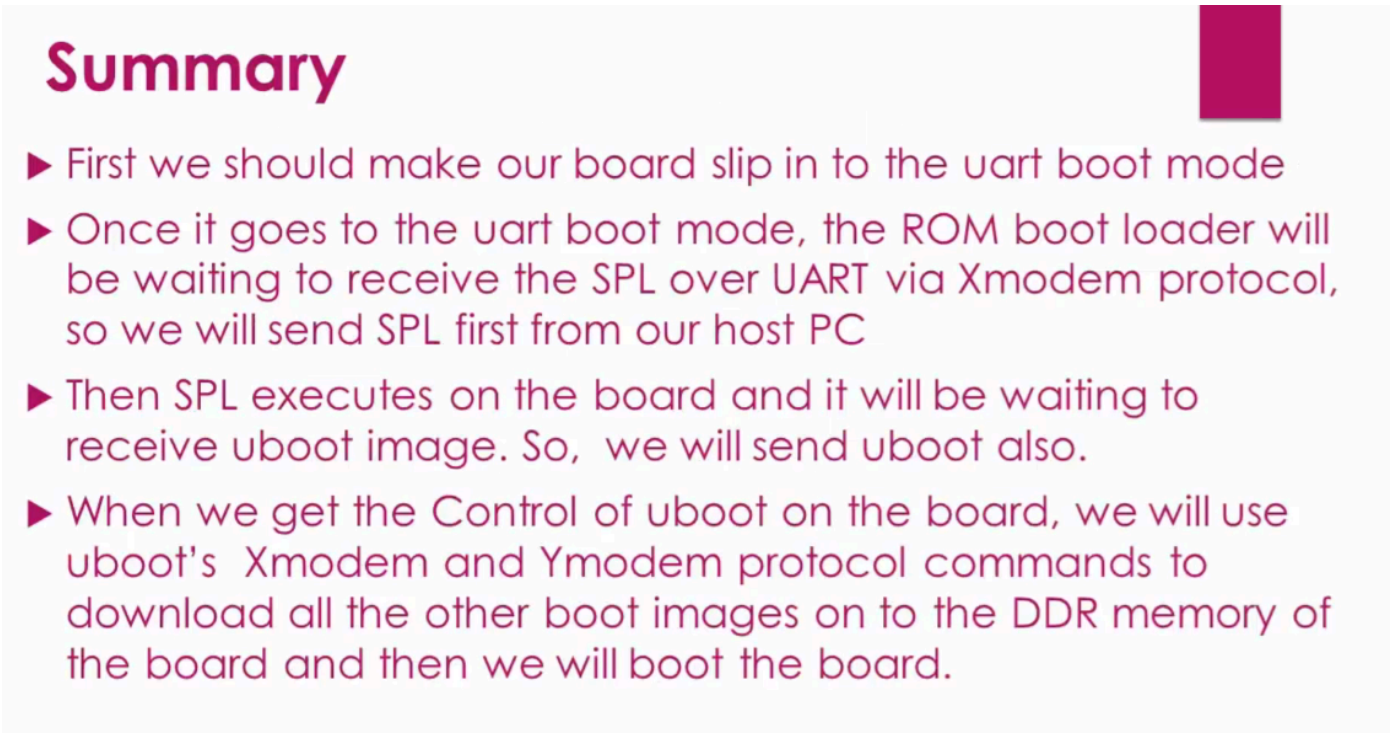
U-Boot'a geldikten sonra kernel, DTB ve dosya sistemini RAM'e (DDR, `0x80000000`'dan başlar) yükleyeceğiz. Dosyaların birbirinin üzerine yazmaması için şu adresler kullanılır:

Dosya	DDR RAM adresi
Linux kernel image (`uImage`)	`0x82000000`
FDT / DTB (`am335x-boneblack.dtb`)	`0x88000000`
RAMDISK / INITRAMFS	`0x88080000`

Seri boot'ta SD kart ve eMMC kullanılmadığı için rootfs'i bağlayacak bir disk yoktur. Bu yüzden dosya sistemi de RAM'e yüklenir: **RAMDISK / INITRAMFS**, RAM'de duran küçük bir rootfs'tir. DTB'ye ayrılan alan `0x88000000` – `0x8807FFFF` arası (512 KB) olduğu için initramfs hemen arkasından, `0x88080000`'dan başlar.

---

6. Özet: Seri Boot Adımları




```
Bilgisayar (HOST)                          BeagleBone Black
─────────────────                          ──────────────────────────────────────

                                           ROM kodu (S2 basılı, SD kart yok) → UART0
u-boot-spl.bin ─── Xmodem ──────────────►  SPL çalışır (DDR'ı hazırlar)
u-boot.img     ─── Ymodem ──────────────►  U-Boot çalışır → U-Boot# komut satırı
uImage         ─── loadx / loady ───────►  0x82000000
.dtb           ─── loadx / loady ───────►  0x88000000
initramfs      ─── loadx / loady ───────►  0x88080000
                                           bootm → Linux açılır
```

1. Önce kartı **UART boot moduna** sokarız (SD kart yok, USB kablosu yok, S2 basılı, adaptörle besleme).
2. UART boot moduna girince ROM kodu, **SPL**'i UART üzerinden **Xmodem** protokolüyle bekler. Bu yüzden bilgisayardan önce SPL'i göndeririz.
3. SPL kartta çalışır (DDR RAM'i hazırlar) ve bu sefer **U-Boot** image'ını bekler. U-Boot'u da göndeririz.
4. U-Boot'un kontrolü ele geçince, U-Boot'un **Xmodem / Ymodem** komutlarıyla (`loadx` / `loady`) diğer bütün boot image'larını (kernel, DTB, initramfs) kartın DDR belleğine indirir ve kartı boot ederiz.

> **Not**
>
> ROM kodu SD karttaki `MLO` dosyasından farklı olarak, UART'tan gelen SPL'in başında **GP header** beklemez. Bu yüzden UART boot'ta `MLO` değil, U-Boot derlemesinin ürettiği ham **`u-boot-spl.bin`** gönderilir.

7. Elimizdeki Boot Dosyaları Nedir?

Seri boot için bilgisayarda (HOST) şu 5 dosya hazır olmalı. Her biri boot zincirinin bir halkasıdır; her dosyanın işi bir sonrakini yükleyip çalıştırmaktır:

Dosya	Nedir?	Kim yükler?	Nereye yüklenir?	Protokol
`u-boot-spl.bin`	**SPL** (Secondary Program Loader). U-Boot'un küçük ilk aşaması. Saatleri, pin ayarlarını, UART'ı ve en önemlisi **DDR RAM**'i hazırlar.	ROM kodu	AM335x'in iç SRAM'i (`0x402F0400` , en fazla ~109 KB)	Xmodem
`u-boot.img`	Asıl **U-Boot**. Başında 64 baytlık U-Boot (mkimage) başlığı vardır. Komut satırını (`U-Boot#`) bize o verir.	SPL	DDR (`0x80800000`)	Ymodem
`uImage`	**Linux kernel**. `zImage` 'in başına U-Boot başlığı (yükleme adresi, giriş noktası, CRC) eklenmiş hali.	U-Boot (`loadx`)	`0x82000000`	Xmodem
`am335x-boneblack.dtb`	**Device Tree Blob**. Kernel'e kartta hangi donanımın olduğunu anlatan derlenmiş dosya.	U-Boot (`loadx`)	`0x88000000`	Xmodem
`initramfs`	RAM'de çalışan küçük **kök dosya sistemi** (genellikle BusyBox). `cpio` arşivi + gzip + U-Boot başlığı. SD/eMMC olmadığı için rootfs'i buradan alırız.	U-Boot (`loadx`)	`0x88080000`	Xmodem

> **Not**
>
> Detaylar: SPL / U-Boot / kernel sırası için booting.md ve linux_booting_process.md, DTB için linux_device_tree.md.

Bu dosyaları nereden bulurum?

Bu dosyaları **kendin yazmazsın**. Hepsi açık kaynak projelerden (U-Boot, Linux kernel, BusyBox) çıkar; senin işin kaynak kodu indirip ayarlamak ve derlemektir. Üç yol vardır:

Yol	Ne zaman?	Artısı / eksisi
1. Hazır (prebuilt) dosyalar	İlk deneme, "kart çalışıyor mu?" testi	En hızlısı; ama içinde ne olduğunu sen seçmezsin.
2. Kaynak koddan elle derleme	Öğrenirken (kursta yapılan)	Her adımı görürsün; daha uzun sürer.
3. Buildroot / Yocto	Gerçek projeler	Hepsini tek komutla üretir; ilk derleme uzun sürer.

Yol 1 — Hazır dosyalar

- **Kursun kaynakları:** Kurslar bu dosyaları genelde derlenmiş olarak verir (Udemy ekleri / kursun GitHub reposu). Kursla birebir uyumlu oldukları için ilk seri boot denemesinde en kolayı bunlar.
- **TI Processor SDK (AM335x):** TI'ın sitesinden indirilir. `board-support/prebuilt-images/` klasöründe genellikle `MLO`, `u-boot-spl.bin`, `u-boot.img`, `zImage`, `.dtb` dosyaları ve bir rootfs hazır gelir.
- **BeagleBoard.org Debian imajı:** Bootloader, kernel ve DTB içerir ama hazır initramfs yoktur; seri boot için daha az uygundur.

Yol 2 — Kaynak koddan derleme

Önce çapraz derleyiciyi (cross compiler) ve `mkimage` aracını kur:

```
sudo apt install gcc-arm-linux-gnueabihf u-boot-tools
```

Dosya	Kaynak kod	Çıktının yeri
`u-boot-spl.bin`, `u-boot.img`	**U-Boot** (`source.denx.de/u-boot` veya GitHub aynası)	`spl/u-boot-spl.bin`, `u-boot.img`
`uImage`	**Linux kernel** (BBB için genelde `github.com/beagleboard/linux`)	`arch/arm/boot/uImage`
`am335x-boneblack.dtb`	Aynı kernel kaynağı	`arch/arm/boot/dts/am335x-boneblack.dtb`
`initramfs`	**BusyBox** (`busybox.net`) ile hazırlanan rootfs klasörü	Senin oluşturduğun dosya

```
# U-Boot  -> spl/u-boot-spl.bin ve u-boot.img
make CROSS_COMPILE=arm-linux-gnueabihf- am335x_evm_config     # yeni sürümlerde: am335x_evm_defconfig
make CROSS_COMPILE=arm-linux-gnueabihf-

# Kernel + DTB  -> arch/arm/boot/uImage ve .../dts/am335x-boneblack.dtb
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs LOADADDR=0x80008000

# BusyBox  -> _install/ klasörü (rootfs'in temeli)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=../rootfs install

# initramfs: rootfs klasöründen cpio arşivi + U-Boot başlığı
cd rootfs
find . | cpio -H newc -o | gzip -9 > ../initramfs.cpio.gz
cd ..
mkimage -A arm -O linux -T ramdisk -C gzip -n initramfs -d initramfs.cpio.gz initramfs
```

Son satırdaki `mkimage` önemli: `bootm` komutu ramdisk'in başında **U-Boot başlığı** görmek ister. Ham `initramfs.cpio.gz` gönderirsen `Wrong Ramdisk Image Format` hatası alırsın (bkz. Bölüm 9-g).

Yol 3 — Buildroot ile hepsini tek seferde

Buildroot toolchain'i, U-Boot'u, kernel'i, DTB'yi ve rootfs'i senin yerine indirip derler:

```
git clone https://gitlab.com/buildroot.org/buildroot.git
cd buildroot
make beaglebone_defconfig
make menuconfig
make                      # ilk derleme 30 dk - 1 saat sürebilir
ls -1 output/images/
```

`make menuconfig`'te seri boot için iki ayar yap:

Menü	Ayar	Neden?
Bootloaders → U-Boot → U-Boot SPL binary name	`MLO spl/u-boot-spl.bin`	Varsayılan sadece `MLO`'yu kopyalar; seri boot için ham SPL gerekir.
Filesystem images → cpio the root filesystem → **Create U-Boot image of the root filesystem**	Aç	`rootfs.cpio.uboot` üretir: başında U-Boot başlığı olan initramfs.

```
mehmet@ubuntu: ~/buildroot
mehmet@ubuntu:~/buildroot$ make beaglebone_defconfig
#
# configuration written to /home/mehmet/buildroot/.config
#
mehmet@ubuntu:~/buildroot$ make
...
mehmet@ubuntu:~/buildroot$ ls -1 output/images/
am335x-boneblack.dtb
MLO                          <-- SD karttan boot için
rootfs.cpio
rootfs.cpio.uboot            <-- seri boot: initramfs
rootfs.ext2
sdcard.img
u-boot-spl.bin               <-- seri boot: SPL
u-boot.img                   <-- seri boot: U-Boot
zImage                       <-- mkimage ile uImage yapılır
```

Buildroot kernel'i `zImage` olarak verir. `bootm` için `uImage`'e çevir (ya da `uImage` yerine `zImage` gönderip `bootm` yerine `bootz` kullan):

```
mkimage -A arm -O linux -T kernel -C none -a 0x80008000 -e 0x80008000 -n Linux -d zImage uImage
```

Her projede baştan mı derleyeceğim?

Hayır. Hangi dosyanın ne zaman yeniden üretildiği:

Dosya	Ne zaman yeniden derlenir?
SPL + U-Boot	Neredeyse hiç. Bir kez derlenir, uzun süre aynısı kullanılır.
Kernel (`uImage`)	Yeni bir sürücü eklendiğinde veya kernel ayarı değiştiğinde.
DTB	Donanım ayarı (pin, I2C/SPI cihazı vb.) değiştiğinde; sadece `make dtbs` yeterli.
initramfs / rootfs	En sık değişen kısım; kendi uygulamanı buraya koyarsın. Sadece `cpio` + `mkimage` adımı tekrarlanır.

Gerçekten **senin yazdığın** kısım: kendi uygulaman, gerekirse bir kernel sürücüsü ve DTS'teki küçük değişiklikler. Geri kalanı hazır projelerin derlenmesidir.

> **İpucu**
>
> Önce kursun hazır dosyalarıyla seri boot'u bir kez çalıştır. Sonra dosyaları **teker teker** kendi derlediklerinle değiştir. Bir şey bozulursa hangi dosyada olduğunu hemen anlarsın.

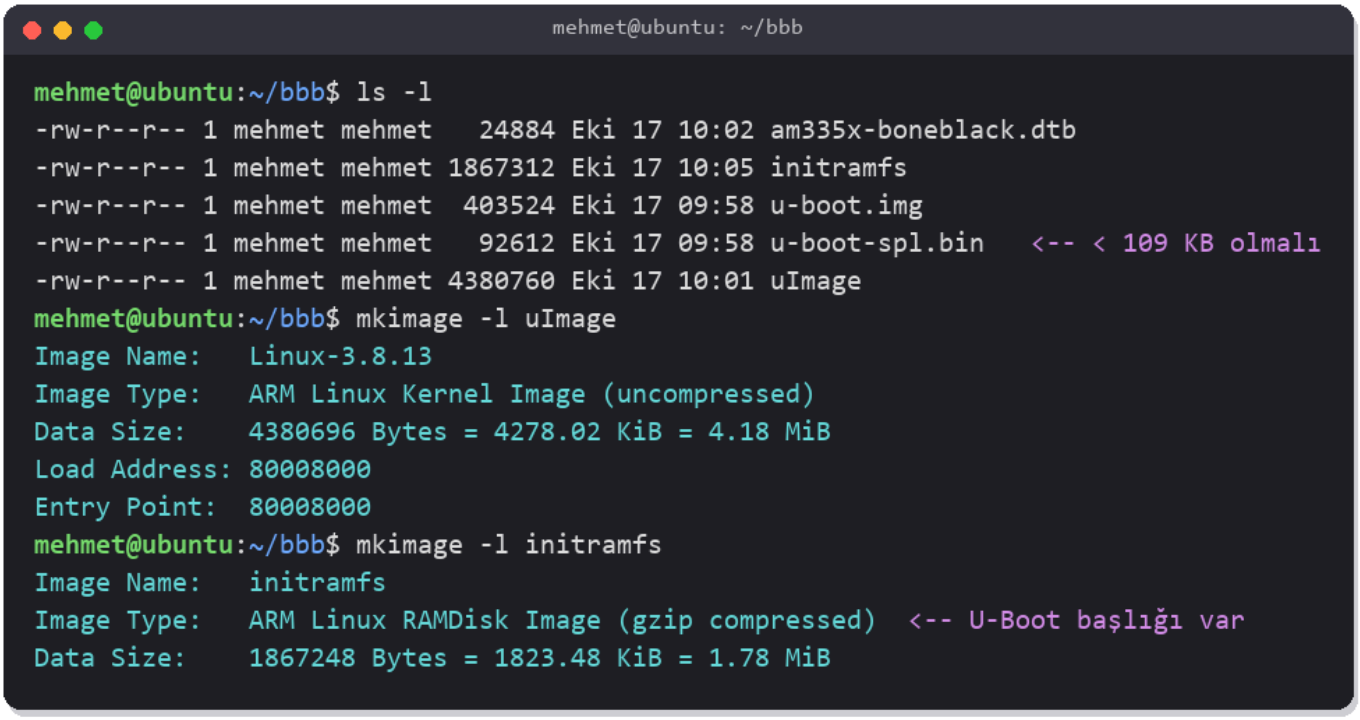
---

8. Adım Adım Seri Porttan Boot

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; sürüm numaraları, tarih, boyut ve hız değerleri senin derlemene göre farklı olacaktır.

Adım 0 — Bilgisayarda hazırlık

Bütün dosyaları tek bir klasörde topla (örnekte `~/bbb`) ve U-Boot başlıklarını `mkimage -l` ile kontrol et:




```
cd ~/bbb
ls -l
mkimage -l uImage
mkimage -l initramfs
```

`u-boot-spl.bin` **109 KB'tan küçük** olmalı; ROM kodu onu iç SRAM'e koyar ve SRAM'de daha fazla yer yoktur.

Adım 1 — Seri kabloyu bağla, Minicom'u hazırla

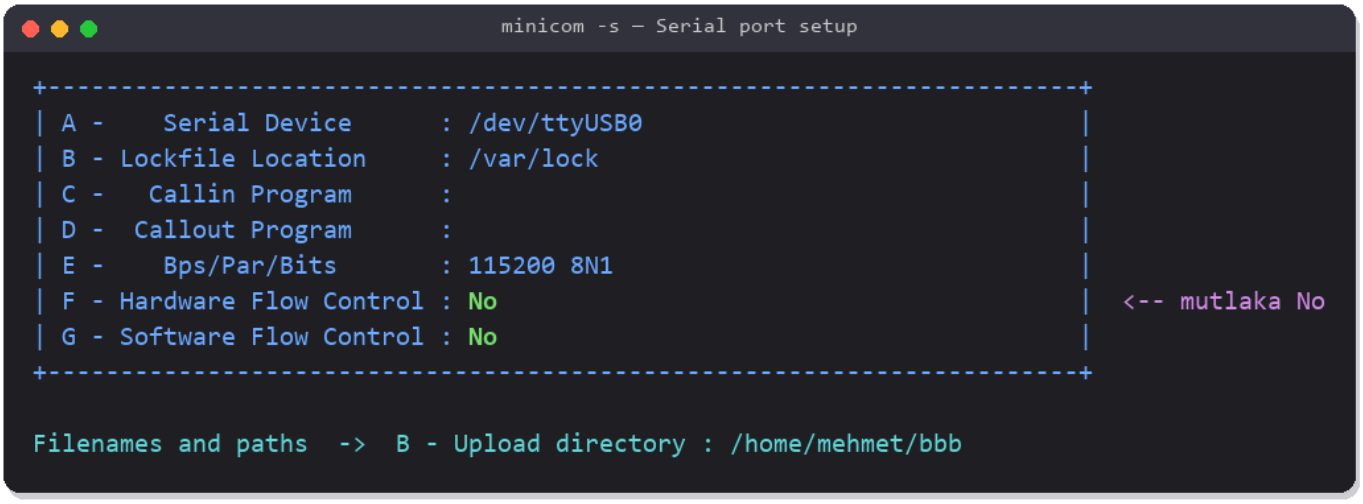
USB-TTL kablosunu J1 header'a bağla (**GND, RX, TX**; kablonun RX'i kartın TX'ine, TX'i kartın RX'ine). Bilgisayarda kablonun hangi aygıt olduğunu bul ve gerekli programları kur:




```
sudo dmesg | tail -3
sudo usermod -aG dialout $USER      # bir kez; sonra oturumu kapatıp aç
sudo apt install minicom lrzsz
minicom -s                          # ayarlar (aşağıda)
cd ~/bbb && minicom -D /dev/ttyUSB0 -b 115200
```

> **Önemli**
>
> Minicom Xmodem/Ymodem gönderirken işi kendisi yapmaz, `lrzsz` paketindeki `sx` (Xmodem) ve `sb` (Ymodem) programlarını çalıştırır. `lrzsz` kurulu değilse dosya gönderilemez.

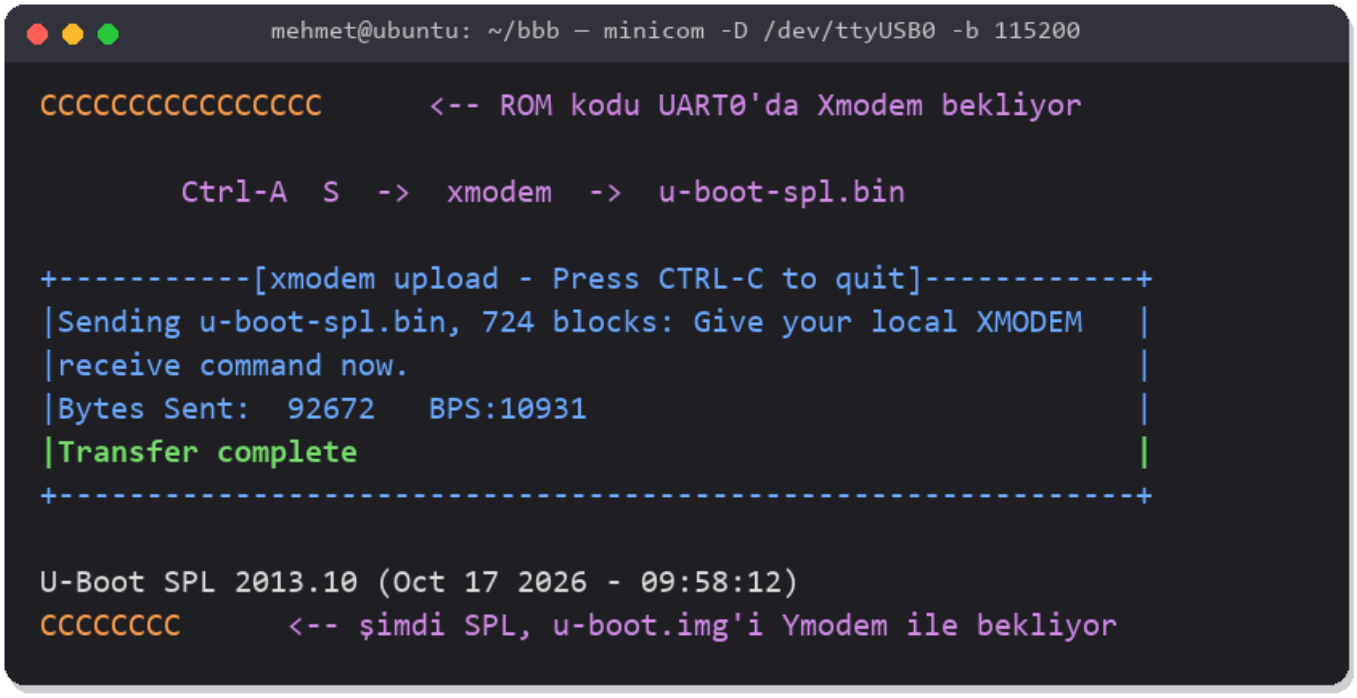
`minicom -s` → **Serial port setup** ekranında ayarlar şöyle olmalı. **Filenames and paths** menüsünde upload klasörünü `~/bbb` yaparsan dosya seçerken her seferinde klasör aramazsın:



Minicom kullanımı için ayrıca: minicom.md

Adım 2 — Kartı UART boot modunda aç ve SPL'i gönder (Xmodem)

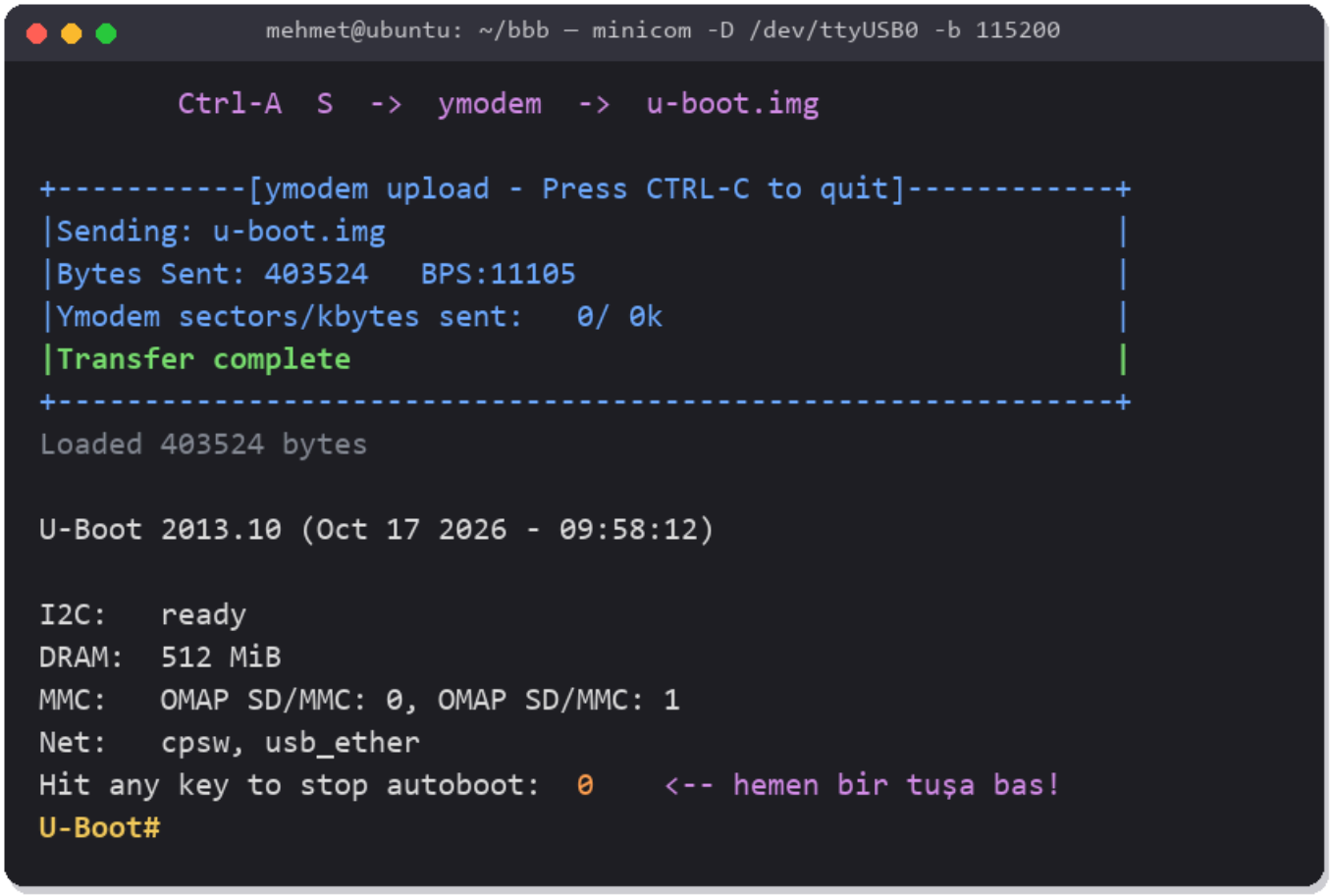
1. SD kartı çıkar, P4 mini USB kablosunu çıkar, 5V adaptörü hazırla (bkz. Bölüm 3).
2. Minicom açıkken **S2**'ye basılı tut, adaptörü tak / **S3**'e bas, sonra S2'yi bırak.
3. Ekranda `CCCC...` akmaya başlar: ROM kodu UART0'da Xmodem ile dosya bekliyor.
4. Minicom'da `Ctrl-A` → `S` → **xmodem** → `u-boot-spl.bin` seç, gönder.



Transfer bitince SPL çalışır, DDR'ı hazırlar, `U-Boot SPL ...` yazısını basar ve tekrar `CCCC` basmaya başlar. Bu sefer bekleyen **SPL**'dir ve **Ymodem** ister.

Adım 3 — U-Boot'u gönder (Ymodem)

1. `Ctrl-A` → `S` → **ymodem** → `u-boot.img` seç, gönder.
2. U-Boot açılır. `Hit any key to stop autoboot` çıkınca **hemen bir tuşa bas.**



> **Uyarı**
>
> Autoboot'u durdurmazsan U-Boot kendi `bootcmd`'ini çalıştırır ve kartın **eMMC**'sindeki Linux'u (ör. Debian) açar. O zaman seri boot'un amacı boşa gider; kartı kapatıp Adım 2'den başla.

Adım 4 — Kernel'i RAM'e al: `loadx 0x82000000`

```
U-Boot# loadx 0x82000000
```

U-Boot Xmodem ile beklemeye başlar. Minicom'da `Ctrl-A` → `S` → **xmodem** → `uImage`.

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# loadx 0x82000000
## Ready for binary (xmodem) download to 0x82000000 at 115200 bps...
C
        Ctrl-A  S  ->  xmodem  ->  uImage
+-----------[xmodem upload - Press CTRL-C to quit]------------+
|Sending uImage, 34225 blocks: Give your local XMODEM receive |
|command now.                                                 |
|Bytes Sent:4380800   BPS:11012                               |  <-- ~6.5 dk sürer
|Transfer complete                                            |
+-------------------------------------------------------------+
## Total Size      = 0x0042d880 = 4380800 Bytes
## Start Addr      = 0x82000000
U-Boot#
```

> **Not**
>
> 115200 baud'da hız en fazla ~11 KB/s'dir. 4 MB'lık bir `uImage` yaklaşık **6–7 dakika** sürer; bu normaldir. Xmodem son paketi 128 bayta tamamladığı için `Total Size` dosya boyutundan biraz büyük görünür, sorun değil.

Adım 5 — DTB ve initramfs'i RAM'e al

```
U-Boot# loadx 0x88000000        (Minicom: xmodem -> am335x-boneblack.dtb)
U-Boot# loadx 0x88080000        (Minicom: xmodem -> initramfs)
```

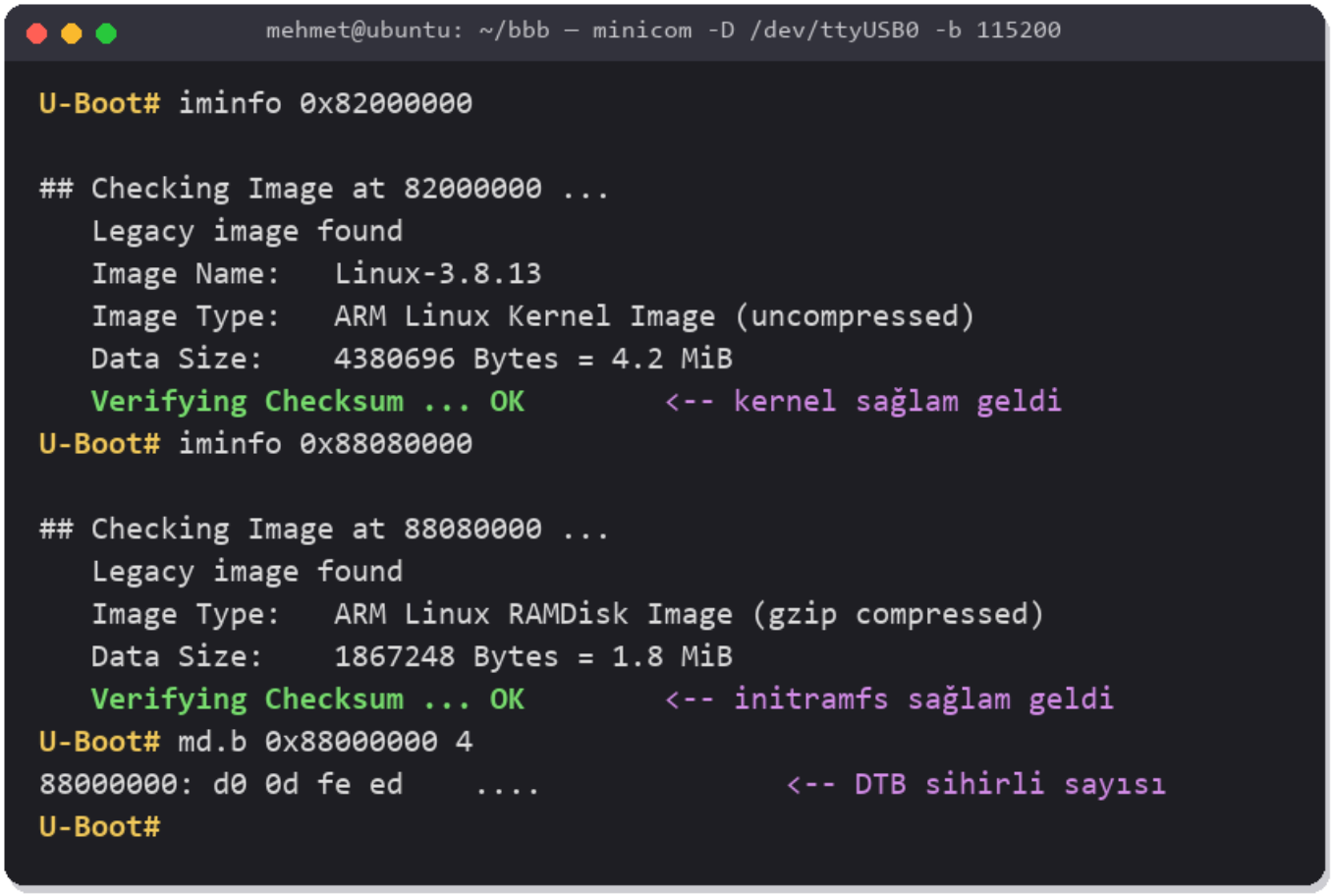
```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# loadx 0x88000000
## Ready for binary (xmodem) download to 0x88000000 at 115200 bps...
        Ctrl-A  S  ->  xmodem  ->  am335x-boneblack.dtb
|Transfer complete                                          |
## Total Size      = 0x00006180 = 24960 Bytes
## Start Addr      = 0x88000000
U-Boot# loadx 0x88080000
## Ready for binary (xmodem) download to 0x88080000 at 115200 bps...
        Ctrl-A  S  ->  xmodem  ->  initramfs
|Transfer complete                                          |
## Total Size      = 0x001c7e80 = 1867392 Bytes
## Start Addr      = 0x88080000
U-Boot#
```

Adım 6 — Yüklenenleri kontrol et (isteğe bağlı ama önerilir)

Boot etmeden önce dosyaların sağlam geldiğini görmek, sonradan "neden açılmadı?" diye uğraşmaktan iyidir:

```
U-Boot# iminfo 0x82000000
U-Boot# iminfo 0x88080000
U-Boot# md.b 0x88000000 4
```



- `iminfo`, U-Boot başlığındaki CRC'yi RAM'deki veriyle karşılaştırır. `Verifying Checksum ... OK` görmelisin.
- DTB'nin U-Boot başlığı yoktur; ilk 4 baytı her zaman `d0 0d fe ed` olmalıdır.

Adım 7 — `bootargs` ayarla ve `bootm` ile boot et

```
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
U-Boot# bootm 0x82000000 0x88080000 0x88000000
```

Boot etmeden önce kernel'e **rootfs'in nerede olduğunu** `bootargs` ile bildiriyoruz:

Parametre	Anlamı
`console=ttyO0,115200`	Kernel mesajlarını UART0'a, 115200 baud ile bas (kernel 3.8 için `ttyO0`; bkz. Bölüm 9-h).
`root=/dev/ram0`	Kök dosya sistemi (rootfs) bir disk değil, **RAM**'de duruyor.
`rw`	Rootfs okunur-yazılır bağlansın.
`initrd=0x88080000`	initramfs'i RAM'de **`0x88080000`** adresine yüklediğimizi kernel'e bildirir (Adım 5'teki `loadx` adresiyle aynı olmalı).
`bootm <kernel> <initramfs> <dtb>`	Sıra önemli: **kernel – ramdisk – DTB**. Ramdisk yoksa ortaya `-` yazılır (`bootm 0x82000000 - 0x88000000`).

> **Not**
>
> `bootm`'in ikinci parametresi (`0x88080000`) de initramfs'in adresini ve boyutunu kernel'e DTB'nin `/chosen` düğümü üzerinden bildirir. Yani adres iki yerden veriliyor; `bootargs`'taki `initrd=` ile `bootm`'deki adresin **aynı** olduğuna dikkat et. `printenv bootargs` ile yazdığını kontrol edebilirsin.

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
U-Boot# bootm 0x82000000 0x88080000 0x88000000
## Booting kernel from Legacy Image at 82000000 ...
   Image Name:   Linux-3.8.13
   Verifying Checksum ... OK
## Loading init Ramdisk from Legacy Image at 88080000 ...
   Image Type:   ARM Linux RAMDisk Image (gzip compressed)
   Verifying Checksum ... OK
## Flattened Device Tree blob at 88000000
   Booting using the fdt blob at 0x88000000
   Loading Kernel Image ... OK
   Loading Ramdisk to 8fe38000, end 8ffffdf0 ... OK
   Loading Device Tree to 8fe2e000, end 8fe37133 ... OK

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 3.8.13 (mehmet@ubuntu) (gcc 4.9) #1 SMP
[    0.000000] Kernel command line: console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
[    1.732050] Trying to unpack rootfs image as initramfs...  <-- initramfs
[    2.104512] Freeing unused kernel memory: 380K
```

Adım 8 — Linux açıldı

Kernel initramfs'i açar, içindeki `/init` (BusyBox) çalışır ve seri konsoldan kabuk gelir:

BusyBox (initramfs) — seri konsol

```
Please press Enter to activate this console.
/ # cat /proc/cmdline
console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
/ # mount
rootfs on / type rootfs (rw)        <-- rootfs RAM'de
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
/ # uname -a
Linux (none) 3.8.13 #1 SMP Sat Oct 17 10:01:44 2026 armv7l GNU/Linux
/ # free -m
             total       used       free     shared    buffers
Mem:           497         14        483          0          0
```

`mount` çıktısında `/`'nin **`rootfs`** olduğunu görürsün: Kart ne SD karta ne eMMC'ye dokunmadan, tamamen seri porttan gelen dosyalarla RAM'den çalışıyor.

> **İpucu**
>
> RAM'deki her şey kart kapanınca silinir. Bir sonraki açılışta bütün adımları (SPL'den itibaren) tekrar yapman gerekir.

Komutların kısa özeti

```
[Kart: S2 basılı + güç, ekranda CCCC]
Minicom: Ctrl-A S -> xmodem -> u-boot-spl.bin
Minicom: Ctrl-A S -> ymodem -> u-boot.img
[Hit any key -> tuşa bas]
U-Boot# loadx 0x82000000          -> xmodem -> uImage
U-Boot# loadx 0x88000000          -> xmodem -> am335x-boneblack.dtb
U-Boot# loadx 0x88080000          -> xmodem -> initramfs
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
U-Boot# bootm 0x82000000 0x88080000 0x88000000
```

---

9. Karşılaşılabilecek Problemler ve Çözümleri

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# bootm 0x82000000 0x88080000 0x88000000
## Loading init Ramdisk from Legacy Image at 88080000 ...
Wrong Ramdisk Image Format           <-- initramfs'te U-Boot başlığı yok
Ramdisk image is corrupt or invalid
U-Boot# iminfo 0x82000000
## Checking Image at 82000000 ...
   Verifying Checksum ... Bad Data CRC  <-- transfer bozuk / adres çakışması

Starting kernel ...
    <-- sonrası boş: console= yanlış ya da DTB uyumsuz

[    2.11] VFS: Cannot open root device "ram0" or unknown-block(1,0)
[    2.11] Kernel panic - not syncing: VFS: Unable to mount root fs  <-- /init yok

[    2.20] Failed to execute /init.  Attempting defaults...    <-- ARM değil / kütüphane eksik
```

a) Ekranda hiç `CCCC` çıkmıyor

Olası neden	Çözüm
SD kart takılı	ROM önce MMC0'ı dener, SD'de MLO varsa oradan açılır. SD kartı çıkar.
P4 mini USB kablosu takılı	ROM USB0'da 4.5 dakika bekler. Kabloyu çıkar, kartı 5V adaptörle besle (Bölüm 3).
S2 basılı değildi	Kart eMMC'den açılır (ekranda eMMC'deki U-Boot/Linux çıkar). Kartı kapat, S2 basılıyken tekrar aç.
TX / RX ters bağlı ya da GND bağlı değil	Kablonun TX'i → kartın RX'i, RX'i → kartın TX'i. GND mutlaka bağlı olmalı.
Yanlış port / baud	`dmesg` ile doğru `/dev/ttyUSBx`'i bul; ayar 115200 8N1 olmalı.
Minicom'u kart açıldıktan sonra açtın	`C`'leri kaçırmış olabilirsin. Önce Minicom'u aç, sonra kartı aç.

b) `CCCC` var ama transfer başlamıyor / yarıda kesiliyor

Olası neden	Çözüm
Hardware Flow Control: Yes	Minicom karta veri göndermez. `minicom -s` ile **No** yap (en sık yapılan hata!).
`lrzsz` kurulu değil (`sx: not found`)	`sudo apt install lrzsz`

Olası neden	Çözüm
Dosya seçmek çok uzun sürdü, ROM vazgeçti	Upload klasörünü önceden ayarla, `C` görünür görünmez gönder. `C` akışı durduysa kartın gücünü kesip S2 ile baştan aç.
Yanlış protokol	ROM ve `loadx` → **xmodem**; SPL'in `u-boot.img` için beklediği ve `loady` → **ymodem**. Karıştırılırsa transfer hiç başlamaz veya `Transfer incomplete` olur.
Transfer sırasında klavyeye basıldı	Gelen tuşlar Xmodem paketlerini bozar. Transfer bitene kadar dokunma.

c) SPL gönderildi ama hiçbir şey olmuyor

Olası neden	Çözüm
`u-boot-spl.bin` yerine `MLO` gönderildi	MLO'nun başında GP header var; ROM onu kod sanıp çalıştırır ve kilitlenir. UART boot'ta **ham** `u-boot-spl.bin` gönderilir.
SPL 109 KB'tan büyük	ROM iç SRAM'e sığdıramaz. SPL'deki gereksiz özellikleri kapatıp yeniden derle.
SPL farklı bir kart/işlemci için derlenmiş	BBB için uygun config (`am335x_evm_config`) ile yeniden derle.

d) SPL çalışıyor ama `u-boot.img` yüklenmiyor

Olası neden	Çözüm
SPL, UART'tan Ymodem alma desteğiyle derlenmemiş (`CONFIG_SPL_YMODEM_SUPPORT`)	SPL başka bir aygıt (MMC) arar ve hata verir. Config'te YMODEM desteğini aç, yeniden derle.
`u-boot.img` Xmodem ile gönderildi	SPL Ymodem bekler; **ymodem** seç.
`u-boot.img` yerine `u-boot.bin` gönderildi	SPL, başında mkimage başlığı olan `u-boot.img` 'i bekler.

e) U-Boot açıldı ama kendi kendine eMMC'den boot etti

`Hit any key to stop autoboot` sırasında tuşa basılmadı. Kartı kapatıp baştan başla; geri sayım başlar başlamaz bir tuşa bas.

> **Dikkat**
>
> Seri boot sırasında U-Boot'ta `saveenv` **yapma**. Değişkenler eMMC'deki ortama yazılır ve kartın normal açılışını bozabilir.

f) Transfer çok yavaş

115200 baud'da ~11 KB/s normaldir; `uImage` 6–7 dakika, initramfs birkaç dakika sürer. Hızlandırmak için:

- **Baud yükseltme:** `loadx` / `loady` ikinci parametre olarak baud alır: `loady 0x82000000 460800`. U-Boot hızı değiştirmeni ister; Minicom'da `Ctrl-A` → `P` ile aynı hıza geç, Enter'a bas, gönder. Bitince U-Boot 115200'e dönmeni ister; Minicom'u geri al ve **ESC**'ye bas. (Her USB-TTL kablo yüksek hızlarda güvenilir çalışmaz; hata alırsan 115200'e dön.)
- **Ymodem:** `loady` 1 KB'lık paketlerle çalıştığı için `loadx`'ten biraz daha hızlıdır.
- **TFTP:** U-Boot açıldıktan sonra Ethernet kablosu varsa `tftpboot` ile saniyeler içinde yüklenir (bkz. uboot_uEnv_ve_seri_yukleme.md).

g) `bootm` hataları

Hata	Neden	Çözüm
`Wrong Ramdisk Image Format` / `Ramdisk image is corrupt or invalid`	initramfs'in başında U-Boot başlığı yok veya yanlış adrese yüklendi	Bilgisayarda `mkimage -T ramdisk ...` ile başlık ekle (Bölüm 7). `iminfo 0x88080000` ile kontrol et.
`Bad Data CRC`	Transfer sırasında veri bozuldu ya da bir dosya diğerinin üstüne yazıldı	Dosyayı tekrar gönder. Adresleri Bölüm 5'teki gibi kullan; DTB 512 KB'ı aşmamalı.
`Bad Magic Number` / `Wrong Image Format for bootm command`	`uImage` yerine `zImage` gönderildi veya adres yanlış	Kernel'i `make uImage LOADADDR=0x80008000` ile derle ya da `zImage` için `bootz` kullan.
DTB bulunamadı / `FDT_ERR_BADMAGIC`	DTB bozuk ya da yanlış adrese yüklendi	`md.b 0x88000000 4` → `d0 0d fe ed` görmelisin. Görmüyorsan DTB'yi tekrar gönder.

h) `Starting kernel ...` yazıp donuyor

Olası neden	Çözüm
`console=` yanlış	Kernel 3.8 (TI omap-serial sürücüsü) → `ttyO0`. Daha yeni (4.x ve sonrası) kernel'lerde genellikle `ttyS0`. Kernel aslında açılmış olabilir, sadece mesajları başka yere basıyordur.
DTB, kernel sürümüyle uyumsuz	DTB'yi **aynı kernel kaynağından** derlenmiş olanla değiştir; BBB için `am335x-boneblack.dtb` (`am335x-bone.dtb` değil).

Olası neden	Çözüm
`bootm` parametre sırası yanlış	Sıra: `bootm <kernel> <ramdisk> <dtb>`.

i) Kernel açılıyor ama panic veriyor

Hata	Neden	Çözüm
`VFS: Unable to mount root fs on unknown-block(1,0)`	initramfs içinde `/init` yok; kernel `root=/dev/ram0`'ı bağlamaya çalışıyor ve başarısız oluyor	rootfs klasöründe `ln -s sbin/init init` yapıp initramfs'i yeniden oluştur, ya da `bootargs`'a `rdinit=/sbin/init` ekle.
`VFS: Unable to mount root fs on unknown-block(0,0)`	Kernel ramdisk'i hiç almamış veya `CONFIG_BLK_DEV_INITRD` kapalı	`bootm`'in ikinci parametresini kontrol et; kernel config'te initramfs/initrd desteğini aç.
`Failed to execute /init` / `No working init found`	initramfs'teki programlar ARM için derlenmemiş veya kütüphaneler eksik	BusyBox'ı `CROSS_COMPILE=arm-linux-gnueabihf-` ile ve tercihen **static** derle; dinamik derlediysen `lib/` klasörüne gerekli kütüphaneleri koy. `file bin/busybox` → `ARM` yazmalı.
`RAMDISK: incomplete write`	initramfs yerine ext2 **initrd** imajı kullanılıyor ve kernel'in varsayılan ramdisk boyutundan büyük	`bootargs`'a `ramdisk_size=<KB>` ekle.

j) Ekranda bozuk / anlamsız karakterler

Baud rate uyuşmuyor. Minicom 115200 8N1 olmalı. Baud yükseltme (f maddesi) denediysen Minicom'u tekrar 115200'e al.